SEMINAR EXPLAINER & VIVA PREP

Decentralized Proof of Personhood

A plain-language guide to what we built, how it works, and why — written so you can explain every piece with confidence.

VTU B.E. Project · SKIT, 2025–26

Team 14 — Revanth, Shashank, Ajay, Chinmayee

Contents

1. The 60-second pitch (say this if you freeze)
2. What problem are we actually solving?
3. The big picture — three pieces, one flow
4. Concepts you must understand before anything else makes sense
5. Walking through the app, step by step
6. A tour of every file, in plain English
7. Why we built it this way (design decisions)
8. What it can't do yet — and why saying so is a good thing
9. Ideas for "future work" (professors always ask)
10. Anticipated questions and how to answer them
11. Glossary — every term in this document, one line each

1. The 60-second pitch

> *Memorize this paragraph. If a professor interrupts you two sentences into your seminar, this is what you fall back on.*

"Online, it's hard to prove you're a real, unique human without handing over your Aadhaar or your face to a company's server. Our project lets someone prove they're a unique human using only their own device — a face-tracking model runs entirely in their browser to check they performed a random physical action like blinking twice — and then a blockchain records that proof as a token in their crypto wallet that can never be sold, transferred, or duplicated. One human, one token, forever. No company holds their video, their photo, or their identity — the blockchain only ever sees a wallet address and a timestamp."

2. What problem are we actually solving?

This category of problem is called **Sybil resistance** — stopping one person from pretending to be a thousand people. It matters everywhere: a single person creating 10,000 fake accounts to farm a token airdrop, bots inflating "likes," or one voter registering under fifty identities in an online poll. Traditional fixes ask you to submit a passport or an Aadhaar number to a central company — which now becomes a single database that can be hacked, sold, or subpoenaed.

Proof of Personhood is the general name for systems that answer "is this a unique real human?" without needing a government ID. Well-known examples your professors may have heard of are **Worldcoin / World ID** (scans your iris with a special hardware orb) and **Humanity Protocol**.

Our project is a lightweight, no-special-hardware version of the same idea: instead of an iris scanner, we use the laptop or phone's own webcam and a free, open, on-device face-tracking model.

3. The big picture — three pieces, one flow

The project has exactly three independently-run pieces of software. Professors like hearing that you can name each piece's *one job* — say it like this:

> **ANALOGY**
>
> Think of it like a nightclub with a bouncer. The **smart contract** is the wristband — once stamped, it can't be peeled off and given to someone else. The **backend** is the bouncer — the only one trusted to actually stamp wristbands, and he checks your ID before doing it. The **frontend** is the queue and the ID-check camera the bouncer uses — what the guest actually sees and interacts with.

PIECE	TECHNOLOGY	ITS ONE JOB
Smart contract	Solidity, deployed on Polygon Amoy (a blockchain test network)	Permanently record "this wallet is verified" and physically prevent the token from ever being transferred.
Backend	Python, FastAPI	The only party allowed to mint tokens. Hands out challenges, checks you're not already verified, saves proof metadata, and signs the mint transaction.
Frontend	React (JavaScript), running in the browser	Everything the user sees and clicks: connect wallet, show the camera, run face-tracking, display the final token.

These three talk to each other over the network — the frontend calls the backend with plain HTTP requests, and the backend calls the smart contract using the blockchain's RPC protocol. No two of them run on the same computer even in production; only the frontend runs on the user's own device.

4. Concepts you must understand before anything else makes sense

> *Read this section slowly once. Everything after this assumes you already know these words.*

4.1 Blockchain, wallet, and "testnet"

A **blockchain** is a shared list of transactions that thousands of independent computers all agree on and keep a copy of — no single company owns or can secretly edit it. A **wallet** (like **MetaMask**,

a browser extension) is just a pair of cryptographic keys: a **public address** (like a bank account number, safe to share — e.g. `0xAbC1...`) and a **private key** (like the PIN — must never be shared, since whoever has it can spend everything in that wallet). We use **Polygon Amoy**, which is a **testnet** — a real blockchain, running the exact same rules as the live "mainnet," except its currency (test MATIC, from a free faucet) is worthless play-money. This lets us build and demo for free without risking real funds.

4.2 Smart contracts and gas

A **smart contract** is a small program that, once deployed to the blockchain, runs exactly as written and can never be secretly changed or turned off by anyone — not even us. Every action that changes the blockchain's state (like minting a token) costs a small fee called **gas**, paid in the network's currency, which is why our contract has a wallet whose only job is paying that fee.

4.3 NFTs and the ERC-721 standard

An **NFT** (Non-Fungible Token) is a token where each individual unit is unique and traceable — unlike a rupee coin, where any ₹10 note is interchangeable with any other. **ERC-721** is the standard rulebook (published as an "Ethereum Request for Comment") that defines exactly how an NFT contract must behave, so wallets and marketplaces everywhere can understand it without custom code. We didn't write our token logic from scratch — we built on top of **OpenZeppelin**, a widely-audited, free library of standard contract building blocks, because writing token-transfer logic from zero is a common source of security bugs.

4.4 Soulbound tokens — EIP-5192

A normal NFT can be sold or given away. A **Soulbound Token (SBT)** is deliberately "glued" to the wallet it was minted to — it can never be transferred, ever. **EIP-5192** is the specific standard that defines how a contract should announce "I am soulbound" so other tools recognize it. This matters here because if a Proof-of-Personhood token could be sold, a "verified human" identity would just become a product for sale — completely defeating the point.

4.5 Liveness detection — why we need it at all

The whole system is worthless if someone can hold up a photo, or play a downloaded video, to the camera and get verified. **Liveness detection** is the general term for any check that the thing in front of the camera is a live, present, three-dimensional human reacting in real time — not a static image or a replayed video. Our method is **challenge-response**: the backend picks a random action ("blink twice") that the user must perform right now, unpredictably, which a static photo can never satisfy.

4.6 MediaPipe, blendshapes, and "on-device" AI

MediaPipe is a free, open-source machine learning toolkit from Google. We use its **Face Landmarker** model, which — given one video frame — outputs two things we actually use: **blendshapes** (52 numbers between 0 and 1, each one measuring how strongly one specific facial muscle is engaged right now — e.g. `eyeBlinkLeft = 0.9` means the left eye is almost fully closed) and a **3-D head rotation matrix** (used to work out if the head is turned left/right or tilted up/down). The critical detail: this model runs **entirely inside the user's own browser**, compiled to WebAssembly and accelerated by the device's GPU. No video frame is ever uploaded anywhere — the browser only ever sends the backend the final result: "challenge passed."

> **ANALOGY**
> It's like a lie-detector built into your own laptop that only ever tells the exam board "pass" or "fail" — the board never gets the actual recording of the test.

4.7 IPFS and Pinata — why not just store data "on the blockchain"

Storing data directly on a blockchain is extremely expensive (you pay gas per byte). **IPFS** (InterPlanetary File System) is a separate, decentralized file-storage network: you upload a file, and get back a fingerprint (a **content hash**) that anyone can use to fetch that exact file from any computer hosting it. **Pinata** is a service that reliably keeps ("pins") your file available on IPFS. So our smart contract only stores a short string — the IPFS address — while the actual verification metadata (when you were verified, which challenge you passed) lives cheaply on IPFS.

4.8 FastAPI, Web3.py, and ethers.js — the "glue" libraries

FastAPI is a Python framework for building web APIs quickly. **Web3.py** is the Python library our backend uses to talk to the blockchain — build a transaction, sign it with the minter's private key, and broadcast it. **ethers.js** is the equivalent library, but for the browser/JavaScript side — it's what lets the React frontend ask MetaMask to connect and later read data straight off the smart contract.

4.9 Environment variables and why `.env` files matter

Secrets like API keys and private keys must never be hard-coded into source files, because source code often ends up on GitHub for the world to see. Instead they're kept in a local `.env` file, loaded at runtime as **environment variables**, and that file is explicitly excluded from Git via `.gitignore`. If a professor asks "how do you protect your keys," this is the answer.

5. Walking through the app, step by step

This is the part to rehearse most, because it's literally what you'll demo live. Each step below names the exact code responsible, so if asked "show me where that happens," you can go straight to it.

Step 1 — Connect wallet

User clicks "Connect MetaMask" (`WalletConnect.jsx`). The app asks MetaMask for permission (`eth_requestAccounts`), then asks it to switch to the Polygon Amoy network — and if the user has never added that network before, it offers to add it automatically (`wallet_addEthereumChain`), so nobody has to manually type in RPC URLs.

Step 2 — Check "have you already got one?"

Before showing the camera, the app reads directly from the smart contract (`contract.isVerified(address)`) — this is a free, instant read, no gas cost — to see if this wallet was already minted a token. If yes, it jumps straight to an "Already Verified" screen instead of wasting the user's time.

Step 3 — Fetch a random challenge

The frontend calls `GET /challenge` on the backend, which picks one at random out of seven (blink, turn head, smile, nod, raise eyebrows, open mouth, look up/down) and returns its text, e.g. *"Blink your eyes twice slowly."*

Step 4 — Run live face-tracking in the browser

The camera opens fullscreen. On every video frame, MediaPipe's Face Landmarker runs and returns blendshape scores and head-rotation angles. A small piece of logic specific to the current challenge (we call it a **detector**, effectively a tiny state machine) watches those numbers: for "blink," it counts how many times the eye-closed score crosses above a threshold and back down; for "turn," it checks the head's left/right rotation angle passed a threshold in each direction and returned to center in between (so a single continuous swipe doesn't wrongly count as two turns). This all happens dozens of times per second, purely on the user's own machine.

Step 5 — Challenge completed → submit

The moment the detector reports success, the camera stops immediately (privacy: nothing is recorded or kept), and the frontend sends `POST /verify` to the backend with only two fields: the wallet address and which challenge was completed. **The video itself is never sent anywhere.**

Step 6 — Backend pins metadata and mints the token

The backend builds a small JSON document (wallet address, timestamp, which challenge was passed, verification method) and uploads it to IPFS via Pinata, getting back an `ipfs://...` address. It then builds a blockchain transaction calling the contract's `mint(walletAddress, ipfsAddress)` function, signs it with the minter wallet's private key, and broadcasts it to Polygon Amoy.

Step 7 — Token appears in the wallet

The frontend shows a success screen with the transaction hash (clickable link to view it on the public block explorer, Polygonscan) and the IPFS metadata link. The user's MetaMask wallet now holds one non-transferable "Proof of Personhood" NFT, forever tied to that address.

6. A tour of every file, in plain English

FILE	WHAT IT IS
`contracts/src/SoulboundToken.sol`	The smart contract. Defines the token, who's allowed to mint (only the contract's owner — our backend's wallet), and blocks every transfer after the initial mint.
`contracts/scripts/deploy.js`	A one-time script: "put the contract onto the Polygon Amoy blockchain and print its address."
`contracts/hardhat.config.js`	Configuration for Hardhat, the toolkit used to compile, test, and deploy the Solidity contract.
`backend/main.py`	The whole backend API — three endpoints: health check, get a random challenge, and verify + mint.
`backend/requirements.txt`	The list of Python libraries the backend depends on.
`frontend/src/App.jsx`	The top-level page. Tracks which "step" the user is on (connect / verify / processing / done) and shows the right screen.
`frontend/src/components/WalletConnect.jsx`	The "Connect MetaMask" screen and network-switching logic.

`frontend/src/components/ChallengeRecorder.jsx`	The biggest, most important file — loads MediaPipe, runs the live camera loop, tracks challenge progress, and submits to the backend.
`frontend/src/components/VerificationBadge.jsx`	The final "Humanity Verified!" success screen, showing the token, transaction link, and IPFS link.
`.env.example` files (three of them)	Templates showing which secret keys each part of the app needs, without containing any real secrets.

7. Why we built it this way (design decisions)

Why split into three separate pieces instead of one program?

Separation of trust. The blockchain enforces the one unbreakable rule ("this token can never move") completely independently of our own servers — even if our backend disappears tomorrow, already-minted tokens stay soulbound forever, because that rule lives in the contract, not in our code. The backend is kept as the single gatekeeper of *minting* because someone has to hold the private key that's allowed to call `mint`, and it's safer for that key to live on a server than inside every user's browser.

Why on-device face-tracking instead of sending video to a server?

Two reasons, and both are strong points to make in your seminar. **Privacy**: a system whose entire purpose is dignity/anti-surveillance shouldn't itself require uploading someone's face video to a server. **Cost and speed**: running detection at 30+ frames per second in the browser is instant and free; sending video to a cloud AI model for every user would be slow and expensive to run at scale.

Why check `isVerified` before minting instead of just trying to mint and catching the error?

A blockchain transaction that fails ("reverts") still costs the minter wallet real gas — it's better to check for free first (a "read" call costs nothing) than to pay to fail.

BE UPFRONT ABOUT THIS IF ASKED

Our project README currently describes an earlier design where video would be analyzed by Google's Gemini AI model on the backend, and a Gemini library is still listed as a dependency. During development we switched to the on-device MediaPipe approach described above for the privacy and cost reasons above, and the README text is left over from the earlier design. If a professor spots this, the honest answer is: "we iterated — we started with server-side AI video

analysis and moved to on-device detection for privacy and performance, and we know the README needs updating to match." That is a completely normal and defensible engineering story.

8. What it can't do yet — and why saying so is a good thing

Professors specifically probe for whether you understand your own system's limits — a team that only lists strengths looks like it doesn't understand its own project. Know these cold:

- **It can be fooled by a very good deepfake or a screen replay of a video.** Blendshape-threshold detection proves "a face performed this expression," not "this is definitely not a screen." Stronger anti-spoofing would need texture/depth analysis or the original server-side video-review approach.
- **Trust is still centralized at the minter wallet.** Whoever holds the backend's private key can mint a token to any address, with or without a real verification — the blockchain guarantees the token can't be re-sold, but it doesn't independently guarantee the mint itself was honest. A more advanced design would have the backend produce a cryptographically signed proof that the contract itself checks, rather than just trusting whoever calls the function.
- **No rate-limiting.** Nothing currently stops someone from spamming the challenge/verify endpoints.
- **No automated tests yet** for the smart contract or backend — a professional version of this project would include tests proving, for example, that a transfer attempt always reverts.

9. Ideas for "future work" (professors always ask)

1. **Signed attestations:** have the backend cryptographically sign proof of verification, and have the smart contract itself verify that signature before minting — removing "just trust the backend" as the security model.
2. **Stronger anti-spoofing:** combine on-device liveness with an occasional server-side video check, or add depth/texture-based deepfake detection.
3. **Move to Ethereum mainnet or a production-grade Layer 2** once audited, instead of a free testnet.
4. **Rate limiting and abuse protection** on the backend API.
5. **Automated test suite** for the contract (e.g., using Hardhat) proving the soulbound guarantee and one-per-wallet rule can never be broken.
6. **A real-world use case demo:** e.g., plug the "isVerified" check into a mock voting app or a mock airdrop claim page, to show the token actually being used for something.

10. Anticipated questions and how to answer them

Q: What stops someone from just showing a photo to the camera?

A: The backend issues a random, unpredictable challenge — like "blink twice" — that a static photo physically cannot perform. The face-tracking model verifies that specific motion happened live, in real time, on the actual video feed.

Q: Where is my face data stored?

A: Nowhere. Face-tracking runs entirely inside the browser on the user's own device. Only the final pass/fail result and the wallet address are ever sent to our server — never the video or any image.

Q: What actually stops the token from being transferred?

A: The smart contract's transfer function is overridden so that any transfer where the token already has an owner is rejected by the blockchain itself — it's not a UI restriction, it's enforced at the protocol level and can't be bypassed even by us.

Q: Why Polygon and not Ethereum mainnet?

A: Polygon Amoy is a free public testnet with the same rules as a production blockchain, letting us build and demo without spending real money on gas fees. A production version would move to a live network after security auditing.

Q: Isn't this centralized, since your backend controls minting?

A: Partially, yes — that's an honest limitation we've identified (see Section 8). Right now the backend is a trusted gatekeeper; a stronger version would use a cryptographically signed attestation that the contract verifies itself, removing the need to trust the backend at all.

Q: What happens if I try to make a second wallet and verify again?

A: Nothing stops a person from owning multiple wallets and getting one token per wallet today — this system proves "this wallet passed a liveness check," not "this human has never done this before across all their wallets," which is a much harder problem every proof-of-personhood project in the world is still working on.

Q: What is gas, and who pays it?

A: Gas is the small fee paid to the blockchain network for processing a transaction. Our backend's minter wallet pays it (in test MATIC from a free faucet) every time a token is minted — the user never pays anything.

Q: Why not store the video or photo on the blockchain as proof?

A: Storing any real data on-chain is extremely expensive, and permanently. We instead store a lightweight text pointer to metadata kept on IPFS, and no video is stored anywhere at all.

11. Glossary — every term in this document, one line each

TERM	MEANING
Blockchain	A shared, tamper-resistant list of transactions kept in sync by many independent computers.
Wallet	A pair of keys (public address + private key) used to own and control blockchain assets.
Testnet	A free practice version of a blockchain, using worthless test currency.
Smart contract	Code deployed to a blockchain that runs exactly as written and can't be secretly altered.
Gas	The fee paid to have a blockchain transaction processed.
NFT	A unique, individually-identifiable token (as opposed to interchangeable currency).
ERC-721	The standard rulebook that defines how NFTs must behave.
Soulbound Token (SBT)	An NFT permanently locked to one wallet — it can never be transferred.
EIP-5192	The standard defining how a contract announces it's soulbound.
Sybil resistance	Preventing one person from posing as many separate identities.
Proof of Personhood	Proving you're a unique real human without revealing a government ID.
Liveness detection	Confirming a live human is present, not a photo, video, or deepfake.
Challenge-response	Asking for a random, unpredictable action that a fake can't pre-prepare for.
MediaPipe	Google's free, open-source toolkit for on-device machine learning models.
Blendshape	A 0–1 score representing how strongly one facial muscle/expression is active.

On-device inference	Running an AI model directly on the user's own hardware, not a server.
IPFS	A decentralized network for storing and fetching files by content fingerprint.
Pinata	A service that keeps files reliably available ("pinned") on IPFS.
FastAPI	A Python framework used to build our backend's web API.
Web3.py / ethers.js	Libraries that let backend/frontend code talk to a blockchain.
MetaMask	A popular browser-extension wallet used to connect to blockchain apps.
Environment variable / .env	A way to keep secret keys out of source code, loaded only at runtime.
RPC	The network protocol used to send requests to a blockchain node.

Prepared as a study aid for the seminar defense. Read Sections 1, 5, and 10 the night before — those three carry the most weight in a live Q&A.